CODE DESIGN

AND

PROPOSED SPARK CONFIGURATIONS

(Alex Chan)

Contents

1. REQUREMENTS 3
1.1 Objectives 3
1.2 Constraints 3
2. DESIGNS 4
2.1 Project Structure Design 4
2.2 Data Pipeline Design 5
2.1.1 Pipeline Stages 6
2.3 Usage of Design Patterns 9
2.3.1 Factory Pattern (RDDIOFactory) 9
2.3.2 Strategy Pattern (via RDDTransformation base class) 9
2.3.3 Chain of Responsibility Pattern (TransformationPipeline) 10
2.3.4 Broadcast Variable Pattern (EnricherTransform) 11
2.4 Other Design Considerations 12
3. PROPOSED SPARK CONFIGURATION 12
3.1 Local Machine Development / Testing 12
3.2 Single-Node / Small Cluster (~1M rows) 13
3.3 Production Cluster (Yarn/Kubernetes, larger dataset) 14

1. REQUREMENTS

1.1 Objectives

Compute the top X most frequently detected item per geographical location from the video camera object detection algorithm in the Town of Utopia.

Inputs:

- **Dataset A (~1M rows)**: detection events with geo location_oid, camera_video_oid, item_name, timestamp, and detection_oid that may contain duplicates.
- **Dataset B (~10K rows)**: lookup table mapping geographical_location_oid to human-readable location names.

Outputs:

- **Per-location ranked list of the X most popular items**, with rank 1 being the most frequently detected item in each geographical location.

1.2 Constraints

1. Implementation must use Spark RDD for transformation logic. DataFrame API is allowed only for reading and writing Parquet.
2. Use reusable design patterns so logic can adapt to other table specs.
3. Code must pass flake8 style checks and follow clean code practices.
4. Consider time and space complexity, minimize shuffle stages and avoid explicit .join if possible.
5. Dataset A contains duplicate detection_oid values. Each detection_oid must count once only.
6. Job's input path for Dataset A & B, and Top X value are configurable and able to change
7. Job's output path can be changed and the output shall save in parquet.
8. Unit test and integration tests shall be included and can be run in local development environment.

2. DESIGNS

2.1 Project Structure Design

The project follows a layered approach with clear separation of concerns (e.g. jobs/, io/, transforms/, util/) such as

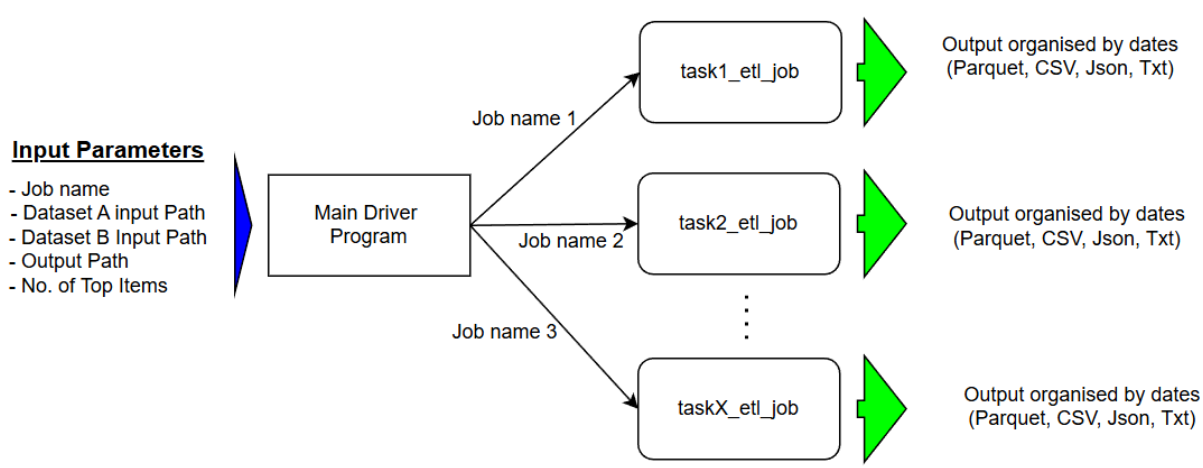
- **Jobs** - Multiple ETL jobs can be supported under the Jobs sub-package and is loaded dynamically.
- **Transforms** - The transformation logics are designed to be self-contained class which can be shared across different jobs
- **IO** - Provide extension and abstractions to the client program to read/write different file formats using factory pattern.
- **Schema** – Column indices are derived from DATASET_SCHEMA at runtime, thus keeping the definition as source of truth.
- **Util** – Utility classes/softwares that are not part of the business logic but serve as helpers and keeping out of concern from the Jobs package.

```
src/item_ranker/
├─ main.py                      Entry point: CLI parsing,
│                                SparkSession, dynamic job module loading
├─ config.py                    Immutable PipelineConfig dataclass
├─ io/
│   ├─ factory_rdd.py           RDDIOFactory: format-agnostic read/write
│   ├─ reader_rdd.py            Concrete readers (Parquet, CSV, JSON, Text)
│   └─ writer_rdd.py            Concrete writers with dated output paths
├─ jobs/
│   ├─ task1_etl_job.py         ETL job (baseline pipeline)
│   ├─ task2_etl_job.py         ETL job with data-skew detection +
│   │                            salted aggregation when skew detected
│   ├─ quality/
│   │   └─ skew_validator.py    Partition-skew detection helper for task2
│   ├─ schema/
│   │   └─ mapping.py           StructType schemas for input/output datasets
│   └─ transforms/
│       ├─ base.py              Abstract RDDTransformation base class
│       ├─ deduplicator.py      Deduplicate by detection_oid
│       ├─ aggregator.py        Count items per (location, item) pair
│       ├─ salted_aggregator.py Two-phase salted aggregation for skew
│       ├─ ranking.py           Rank + top-X filter per location
│       ├─ enricher.py          Broadcast map-side join with location names
│       └─ pipeline.py          TransformationPipeline orchestrator
```

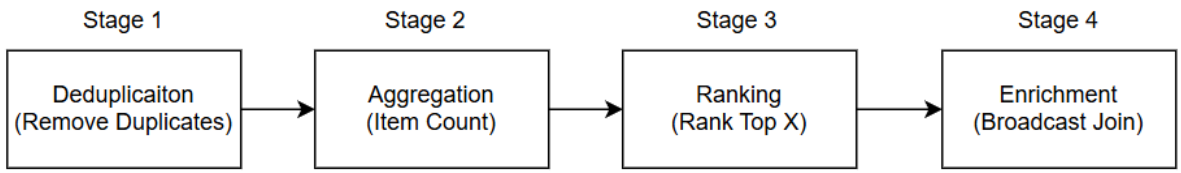
```
└─ util/
    ├─ environment.py           .env loading via python-dotenv
    └─ log_manager.py           Centralized logging configuration

tests/
├─ conftest.py                  Session-scoped local SparkSession fixture
├─ test_environment.py          .env loader unit tests
├─ test_log_manager.py          LogManager unit tests
├─ unit/
│   ├─ test_config.py           PipelineConfig unit tests
│   ├─ test_io_factory.py       RDDIOFactory unit tests
│   ├─ test_main_cli.py         CLI parsing + job-loader unit tests
│   ├─ test_schema_mapping.py   Schema contract unit tests
│   ├─ test_skew_check.py       Skew-detection unit tests
│   ├─ test_writer_dated_path.py Dated-output-path writer tests
│   └─ transforms/              One file per transform class
└─ integration/                 End-to-end pipeline tests on local Spark
    ├─ test_task1_etl_job.py    Synthetic-data full pipeline (task1)
    ├─ test_task2_etl_job.py    Balanced + skewed-data paths (task2)
    ├─ test_main_end_to_end.py  CLI subprocess driving main()
    └─ test_with_real_fixtures.py  Smoke test on data/input/*.parquet
                                     (parametrized for task1 + task2)
```

2.2 Data Pipeline Design

The main program serves as the entry point for the Spark driver program. It is designed to accept a set of input parameters to load the jobs dynamically without modifying the entry point following the **open/closed principle** in practice.



2.1.1 Pipeline Stages



<u>Stage 1: Deduplication (DeduplicatorTransform)</u>

The depuplication is placed in the first stage to ensure data correctness (since it is mentioned data contain duplicates) and reduce the data size so that the downstream operation can be more efficient, e.g. less data to shuffle, memory pressure and faster execution.

Algorithm	Output	Time Complexity
rdd.map(tuple) .map(row -> (row[detection_oid_index], row)) .reduceByKey(lambda a, b: a) .map(kv -> kv[1])	(geographical_location_oid, video_camera_oid, detection_oid, item_name, timestamp_detected)	0(n) time, 1 shuffle stage

Rationale to use reduceByKey over groupByKey or distinct():

- reduceByKey performs map-side combine (local combine) before shuffle, thereby minimizing the data transferred across network
- distinct would treats data as duplicate if all columns are identical. However, the requirement is to consider "detection_oid" as duplicate instead.
- distinct may run the risk of overcount in the event that all columns are the same except for the timestamp.
- groupBykey would incur more network I/O by building the full list in memory by shuffling all rows which is highly inefficient at this stage of the pipeline.

<u>Stage 2: Aggregation (AggregatorTransform) for task1_etl_job</u>

The aggregation is placed in the second stage to perform the aggregation which further reduce to a smaller dataset for efficient downstream operation. In addition, it also provides the count which is needed for the Ranking.

Algorithm	Output	Time Complexity
rdd.map(row -> ((row[geo_oid_index], row[item_name_index]), 1)) .reduceByKey(add)	((geo_oid, item_name), count)	0(n) time, 1 shuffle stage

Rationale to use reduceByKey

- Sums are computed per partition (local combine) before shuffle, thus reduce network I/O significantly compared to use groupByKey and minimize OutOfMemoryError if the key has millions of values (highly skewed)
- Operator(add) can be used safely since it guarantees the result is correct regardless the order of the data arrives at reducers.

<u>Stage 2: Salted_Aggregation (SaltedAggregatorTransform) for task2_etl_job</u>

The salted_aggregation is placed in the second stage (when data skew is detected) to perform the same count aggregation, but with a random salt is appended to each composite key in phase 1 so that the dataset is partitioned across multiple reducer slots instead of a single one, and phase 2 strips the salt and merges the partial counts.

The end result is the same as the aggregation performed and provides the count for the ranking.

Algorithm	Output	Time Complexity
Phase 1: Salted = rdd.map(lambda row: (tuple(row[i] for i in indices) + (random.randint(0, num_salts - 1),), 1,)).reduceByKey(add) Phase 2: salted.map(lambda kv: (kv[0][:-1], kv[1])) .reduceByKey(add)	((geo_oid, item_name), count)	O(n) time, 2 shuffle stages, Phase 2: input size O(|K| * num_salts) where: • |K| = the cardinality (size) of that set — the number of distinct keys. • num_salts = how many salt buckets each key is split into during phase 1.

Rationale to use two-phases approach:

- Without performing salting first, all of the dataset may send to the same reducer, becomes bottleneck and slow down the entire operation.
- Operator(add) in phase 1 adds up the count locally on each partition, thus reduce network I/O before shuffling. The number of rows send are distinct keys * num_salts after salting.
- Phase 2 is identical to the aggregation performed in task1_etl_job by adding the remaining counts (after phase 1 summation) after removing the salt.

Stage 3: Ranking (RankingTransform)

This Ranking is placed in the third stage to sort the count across the entire dataset as per required for the output.

Algorithm	Output	Time Complexity
rdd.map(kv -> (geo_oid, (item_name, count))) .groupByKey() .flatMap(_rank_items: sort desc by count desc, Item name asc, and take top_x)	(geo_oid, rank, item_name)	O(m) time where m is the aggregated item count, 1 shuffle stage. Per-group sorting is O(k log k) per location where: • K = the number of unique item_name values within one geographical_location.

Rationale to use groupByKey

- After aggregation, the dataset is reduced at most ~10K * unique item entries.
- Sorting needs to occur over complete set of items to assign the ranking, thus groupByKey is necessary here.
- Per-group sorting is 0(K log K) where ki is the number of unique items per location.

Stage 4: Enrichment (EnricherTransform)

Enrichment is placed at the last stage since the dataset A is reduced significantly. Thereby, there is fewer rows to enrich, which enable faster compute.

Algorithm	Output	Time Complexity
broadcast_dict = sc.broadcast(locations_dict) rdd.map(row -> insert location_name at insert_pos from broadcast lookup)	(geo_oid, geo_location, rank, item_name)	O(n) time, 0 shuffle stages. Broadcast cost: one-time O(10K) network transfer to each executor

Rationale to use Broadcast join instead of RDD.join

- Dataset B (~10k rows) can easily fit into executor memory
- Map side lookup adds zero shuffle stage
- Reduce the overhead of shuffling and partitioning both RDDs if using standard join

2.3 Usage of Design Patterns

2.3.1 Factory Pattern (RDDIOFactory)

A single factory class dispatches read and write operations to the correct format-specific reader/writer (Parquet, CSV, JSON, Txt). This decouples the ETL logic from the file format and adding new format can be easily catered without affecting the existing ETL logic.

```
# read the parquet file using rdd factory
dataset_a_rdd = RDDIOFactory.read_rdd(
        spark, PARQUET_FORMAT, config.dataset_a_path)

RDDIOFactory.write_rdd(
        spark,
        result_rdd,
        PARQUET_FORMAT,
        config.output_path,
        OUTPUT_SCHEMA,
    )
```

2.3.2 Strategy Pattern (via RDDTransformation base class)

Each transformation (deduplication, aggregation, ranking, enrichment) is an independent strategy class implementing the execute(rdd) method. This allows:

- Swap any transform without modifying the rest of the pipeline.
- Support building entirely different pipelines for different specifications by composing different transform objects.

```
class RDDTransformation(ABC):
    """Abstract base class for RDD transformations.
    Enables the Pipeline/Strategy design pattern: each transformation
    is a composable, reusable unit. Swap out transforms to create
    entirely different pipelines for different specifications.
    """
    @abstractmethod
    def execute(self, rdd: RDD):
        """Apply the transformation to an RDD and return a new RDD."""
        raise NotImplementedError

class DeduplicatorTransform(RDDTransformation):
    """Deduplicate rows by a specified key index.

    Given an RDD of tuples, keys by the element at `key_index`,
    keeps only one row per unique key via reduceByKey, then
```

```
    strips the key wrapper to return the original tuple structure.

    Args:
        key_index: Positional index of the deduplication key in each tuple.
    """

    def __init__(self, key_index: int):
        self._key_index = key_index

    def execute(self, rdd):
        """Deduplicate RDD rows by the configured key index."""
        return (
            rdd
            .map(tuple)
            .map(lambda row: (row[self._key_index], row))
            .reduceByKey(lambda a, b: a)
            .map(lambda kv: kv[1])
        )
```

2.3.3 Chain of Responsibility Pattern (TransformationPipeline)

The pipeline orchestrator accepts a list of RDDTransformation objects and applies them sequentially. This helps to:

- Adding/removing/reordering stages requires only changing the list passed to TransformationPipeline, not modifying any transform class
- Each stage's input/output contract is explicit (RDD of tuples)
- Pipeline is testable at both unit (individual transform) and integration (full chain) levels

Type-checked at construction: non-RDDTransformation objects raise TypeError immediately

```
# configure the transformation pipeline
pipeline = TransformationPipeline([
    DeduplicatorTransform(key_index=detection_idx),
    AggregatorTransform(key_indices=(geo_oid_idx, item_name_idx)),
    RankingTransform(top_x=config.top_x),
    EnricherTransform(
        enrich_data=dataset_b_broadcast,
        pri_key_index=geo_oid_idx,
        insert_pos=geo_loc_idx
    ),
])
# run the transformation pipeline
result_rdd = pipeline.run(dataset_a_rdd)
```

```python
class TransformationPipeline:
    """Orchestrates a sequence of RDD transformations.
    Chains multiple RDDTransformation objects, executing them
    sequentially. This enables reuse: swap transforms to build
    entirely different pipelines for different specifications.
    """
    def __init__(self, transforms: list):
        for t in transforms:
            if not isinstance(t, RDDTransformation):
                raise TypeError(
                    f"Expected RDDTransformation, got {type(t).__name__}"
                )
        self._transforms = list(transforms)

    def run(self, rdd):
        """Execute all transformations in sequence on the given RDD."""
        for transform in self._transforms:
            rdd = transform.execute(rdd)
        return rdd
```

2.3.4 Broadcast Variable Pattern (EnricherTransform)

The Spark broadcast variable pattern is a distributed computing design pattern used to efficiently share large, read-only data across worker node. This is a classic Spark optimization for small-large table joins.

Dataset B is collected into a python dict and broadcast to all executors, and used for map-side lookups.

```python
def broadcast_dataset_b(spark, dataset_b_rdd):
    """Collect locations RDD into a dict and broadcast it.
    Args:
        spark: SparkSession instance.
        data_Set_b_rdd: RDD of
            (geographical_location_oid, geographical_location).
    Returns:
        A Spark broadcast variable wrapping {geo_oid: geo_location} dict.
    """
    locations_dict = dict(
        dataset_b_rdd.map(tuple).map(lambda row: (row[0], row[1])).collect()
    )
```

```
    return spark.sparkContext.broadcast(locations_dict)

# dict {geo_oid: geo_location}
dataset_b_broadcast = broadcast_dataset_b(spark, dataset_b_rdd)
```

2.4 Other Design Considerations

- **Schema** – Column indices are derived from DATASET_SCHEMA at runtime, enabling the transformation logic to fit into different table specs easily.

- **{job_name}/{Dated} Output Path** – The output path is organized according to the {jobname}/{current run date} and not the column timestamp. This is because the timestamp column not defined in the Output schema. This organizing of the output to the jobname and run date allows the data to manage easily (e.g. easily identify which data to archive and determine the freshness of the data)

- **Immutable data class** – A frozen data class ensure all runtime parameter is not accidentally altered during the execution and provides a clean data contract between the CLI and pipeline logic.

3. PROPOSED SPARK CONFIGURATION

3.1 Local Machine Development / Testing

spark.master = local[*]
spark.ui.enabled = true
spark.sql.shuffle.partitions = 2
spark.serializer = org.apache.spark.serializer.KryoSerializer
spark.driver.memory = 1g

Configuration	Rationale
spark.serializer= org.apache.spark.serializer.KryoSerializer	• 2-10x faster than default java serialization
spark.sql.shuffle.partitions= 2	• Min CPU specs for client machine support 2 cores • Data size is not huge
spark.driver.memory= 1g	• Default value which balances speed and resource usage

	• Pipeline is mostly map/reduce with small metadata in memory (example, broadcast for lookup) • Enable to detect memory-pressure issues earlier instead of hiding with a very large heap

3.2 Single-Node / Small Cluster (~1M rows)

```
spark.master                  = local[*] or yarn
spark.serializer              = org.apache.spark.serializer.KryoSerializer
spark.sql.shuffle.partitions  = 64
spark.default.parallelism     = 64
spark.driver.memory           = 2g
spark.executor.memory         = 4g
spark.executor.cores          = 2
spark.rdd.compress            = true
spark.shuffle.compress        = true
spark.shuffle.spill.compress  = true
spark.sql.adaptive.enabled    = true
```

Configuration	Rationale
spark.serializer= org.apache.spark.serializer.KryoSerializer	• 2-10x faster than default java serialization
spark.sql.shuffle.partitions= 64 spark.default.parallelism = 64	• Good starting point (vs default =200) for ~1M rows across a small cluster. • Prevents partitions too large (waste overheads) or too small (memory pressure, stragglers) • Parallelism use the same baseline as partitions will keep the partitioning behavior more predictable across RDD operations and shuffle stages.
spark.driver.memory= 2g spark.executor.memory = 4g	• Demand for driver memory is normally lower than executor memory since it handling task scheduling, DAG construction, and small metadata • Executor memory is typically 2x (common rule of thumb) for processing tasks, caching data, and managing shuffle data.

Configuration	Rationale
	• 4g executor memory can easily accommodate groupByKey for ranking and broadcast variable.
spark.rdd.compress = true spark.shuffle.compress = true spark.shuffle.spill.compress = true	• Compression reduces network I/O at the cost of minor CPU usage, especially for pipelines where shuffles dominate
spark.sql.adaptive.enabled = true	• Adaptive query execution helps to rebalance partitions dynamically in mixed DataFrame/RDD stages (parquet read/write)

3.3 Production Cluster (Yarn/Kubernetes, larger dataset)

```
spark.master                    = yarn / k8s://...
spark.deploy.mode               = cluster
spark.num.executors             = 4-8
spark.executor.cores            = 4
spark.executor.memory           = 8g
spark.driver.memory             = 4g
spark.serializer                = org.apache.spark.serializer.KryoSerializer
spark.sql.shuffle.partitions    = 200
spark.default.parallelism       = 200
spark.rdd.compress              = true
spark.shuffle.compress          = true
spark.shuffle.spill.compress    = true
spark.sql.adaptive.enabled      = true
spark.sql.adaptive.coalescePartitions.enabled = true
spark.speculation               = true
spark.speculation.quantile      = 0.9
spark.dynamicAllocation.enabled = true
spark.dynamicAllocation.minExecutors = 2
spark.dynamicAllocation.maxExecutors = 16
```

Configuration	Rationale
spark.serializer= org.apache.spark.serializer.KryoSerializer	• 2-10x faster than default java serialization

Configuration	Rationale
spark.sql.shuffle.partitions= 200 spark.default.parallelism = 200	• 200 shuffle partitions are the default and standard recommendations for cluster-scale Spark jobs • Parallelism use the same baseline as partitions will keep the partitioning behavior more predictable across RDD operations and shuffle stages
spark.num.executors = 4-8 spark.executor.cores = 4 spark.executor.memory = 8g spark.driver.memory = 4g	• Driver memory of 4g to 8g is typically the standard production settings. • Executor memory is typically 2x (common rule of thumb). • 4 executor.cores are typical setting for production cluster balanced with 4-8 num.executors. This made up of total 32 cores for the cluster.
spark.rdd.compress = true spark.shuffle.compress = true spark.shuffle.spill.compress = true	• Compression reduces network I/O at the cost of minor CPU usage, especially for pipelines where shuffles dominate
spark.sql.adaptive.enabled = true	• Adaptive query execution helps to rebalance partitions dynamically in mixed DataFrame/RDD stages (parquet read/write)
spark.speculation = true spark.speculation.quantile = 0.75 (standard) to 0.9 (highly skewed)	• Speculation helps to relaunch the straggler tasks to meet SLA. • Quantile triggers speculation when the task is slower than 75% or 90% of the peers. • If the data is highly skewed, it is naturally the tasks will be taken a longer time. 0.75 will trigger unnecessary re-runs, wasting cluster resources. Therefore, 0.9 forces Spark to speculate on the last few tasks to finish.
spark.dynamicAllocation.minExecutors = 2 spark.dynamicAllocation.maxExecutors = 16	• Scales executor counts dynamically to actual workload, reducing resource waste during lighter stages